7.1 테스트 구조 분석

개요

Composable Finance 프로젝트는 다층적이고 포괄적인 테스트 전략을 채택하여 코드 품질과 시스템 안정성을 보장합니다. 테스트 구조는 단위 테스트, 통합 테스트, 벤치마크 테스트의 세 가지 주요 계층으로 구성되어 있으며, 각각 특정한 목적과 범위를 가지고 있습니다.

7.1.1 단위 테스트 구조 (`#[cfg(test)]` 모듈)

테스트 모듈 구성

각 팰릿은 표준화된 테스트 구조를 따르며, `#[cfg(test)]` 조건부 컴파일을 통해 테스트 코드를 분리합니다.

Pablo DEX 팰릿 테스트 구조

```
// code/parachain/frame/pablo/src/lib.rs

#[cfg(test)]

mod mock;

#[cfg(test)]

mod test;
```

Pablo 팰릿의 테스트는 다음과 같이 세분화되어 있습니다:

```
code/parachain/frame/pablo/src/test/
```

```
├── mod.rs // 테스트 모듈 진입점
├── common_test_functions.rs // 공통 테스트 유틸리티
├── dual_asset_constant_product_tests.rs // DACP AMM 테스트
├── dual_asset_constant_product_tests_new.rs // 새로운 DACP 테스트
└── pablo_tests.rs // 일반 Pablo 기능 테스트
```

테스트 모듈 구성 예시:

```rust
// code/parachain/frame/pablo/src/test/mod.rs
mod common_test_functions;
mod dual_asset_constant_product_tests;
mod dual_asset_constant_product_tests_new;
mod pablo_tests;
```

Liquid Staking 팰릿 테스트 구조

```rust
// code/parachain/frame/liquid-staking/src/lib.rs
#[cfg(test)]
mod mock;
#[cfg(test)]
mod tests;
```

Liquid Staking 팰릿은 단일 테스트 파일 구조를 사용합니다:

```
code/parachain/frame/liquid-staking/src/
```

```
├── lib.rs // 메인 팰릿 코드
├── mock.rs // 테스트용 런타임 모킹
└── tests.rs // 단위 테스트 구현
```

Mock 런타임 구성

각 팰릿은 독립적인 테스트 환경을 위해 `mock.rs` 파일에서 테스트용 런타임을 구성합니다.

Pablo 팰릿 Mock 구성 예시:

```rust
// code/parachain/frame/pablo/src/mock.rs
use frame_support::{
construct_runtime, parameter_types,
traits::{Everything, GenesisBuild},
};

construct_runtime!(
pub  enum Test where
Block = Block,
NodeBlock = primitives::Block,
UncheckedExtrinsic = UncheckedExtrinsic,
{
System: frame_system,
Assets: pallet_assets,
Pablo: crate,
```

```
// 기타 필요한 팰릿들
}
);
```

테스트 헬퍼 함수

공통 테스트 기능은 별도의 헬퍼 모듈로 분리되어 재사용성을 높입니다.

Pablo 공통 테스트 함수 예시:

```
// code/parachain/frame/pablo/src/test/common_test_functions.rs
pub fn create_test_pool() -> PoolId {
// 테스트용 풀 생성 로직
}

pub fn mint_test_assets(account: AccountId, amount: Balance) {
// 테스트용 자산 발행 로직
}
```

7.1.2 통합 테스트 구조 (`integration-tests/` 디렉토리)

통합 테스트 아키텍처

통합 테스트는 `code/integration-tests/runtime-tests/` 디렉토리에 위치하며, TypeScript를 사용하여 실제 런타임과의 상호작용을 테스트합니다.

```
code/integration-tests/runtime-tests/
```

```
├── src/ // 테스트 유틸리티 및 헬퍼
├── tests/ // 실제 테스트 시나리오
├── util-scripts/ // 유틸리티 스크립트
├── package.json // Node.js 의존성
├── tsconfig.json // TypeScript 설정
└── picasso-types.json // 체인 타입 정의
```

테스트 설정 구성

통합 테스트는 `config.json` 을 통해 다중 체인 환경을 구성합니다:

```
{
"endpoints": {
"picassoEndpoint": "ws://127.0.0.1:9988",
"composableEndpoint": "ws://127.0.0.1:29988",
"kusamaEndpoint": "ws://localhost:9944"
},
"chains": {
"picasso": {
"chainType": "substrate",
"addresses": {
"substrateEscrowAddress": "5xZC6K86GX7WbBv6UD4pLuFXbpUMqdPmMGpoZqij2dGewTDX",
"cosmosEscrowAddress": "5ywFeGCA5MvVqeEmTXshfzPJe9tCXEFDZKCTeoYCYYQWGSd8"
}
```

```
}

}

}
```

멀티홉 테스트 시나리오

크로스체인 기능의 통합 테스트는 복잡한 멀티홉 시나리오를 다룹니다:

```
// code/integration-tests/runtime-tests/tests/multihopTests.ts

describe('Multihop XCM-IBC Tests', () => {

test('Transfer from Picasso to Osmosis via Centauri', async () => {

// 멀티홉 전송 테스트 로직

});

});
```

테스트 환경 구성

통합 테스트는 다음과 같은 체인 환경을 시뮬레이션합니다:

1. **Picasso (Kusama Parachain)**

- 로컬 엔드포인트: `ws://127.0.0.1:9988`
- 체인 타입: Substrate

2. **Composable (Polkadot Parachain)**

- 로컬 엔드포인트: `ws://127.0.0.1:29988`
- 체인 타입: Substrate

3. **Centauri (Cosmos Chain)**

- IBC 연결을 통한 크로스체인 테스트

7.1.3 벤치마크 테스트 (`benchmarking` 모듈)

벤치마크 구조

각 팰릿은 성능 측정을 위한 벤치마크 모듈을 포함합니다:

```
// code/parachain/frame/pablo/src/lib.rs

#[cfg(any(feature = "runtime-benchmarks", test))]

mod benchmarking;
```

Pablo DEX 벤치마크 구현

```
// code/parachain/frame/pablo/src/benchmarking.rs

use frame_benchmarking::{benchmarks, impl_benchmark_test_suite};



benchmarks! {

where_clause {

where T::BlockNumber: From<u32>,

T::Balance: From<u128>,

T::AssetId: From<u128>

}

create {
```

```
let usdc = 100.into();
let usdt = 101.into();
let owner: T::AccountId = whitelisted_caller();
let swap_pool_init = amm_init_config::<T>(
owner,
CurrencyPair { base: usdc, quote: usdt },
Permill::from_percent(50),
Permill::from_percent(1)
);
}: _(RawOrigin::Signed(owner), swap_pool_init)
swap {
// 스왑 벤치마크 구현
}: _(RawOrigin::Signed(caller), pool_id, pair, quote_amount, min_receive)
}
```

벤치마크 헬퍼 함수

벤치마크는 테스트 환경 설정을 위한 헬퍼 함수들을 포함합니다:

```
fn amm_init_config<T: Config>(
owner: T::AccountId,
pair: CurrencyPair<T::AssetId>,
base_weight: Permill,
fee: Permill,
```

```
) -> PoolInitConfigurationOf<T> {
let mut assets_weights = Vec::new();
assets_weights.push((pair.base, base_weight));
assets_weights.push((pair.quote, base_weight.left_from_one()));
PoolInitConfiguration::DualAssetConstantProduct {
owner,
fee,
assets_weights
}
}
```

성능 측정 메트릭

벤치마크는 다음과 같은 성능 지표를 측정합니다:

1. **실행 시간**: 각 extrinsic의 실행 시간
2. **가중치 계산**: 블록 가중치 소비량
3. **스토리지 접근**: 데이터베이스 읽기/쓰기 횟수
4. **메모리 사용량**: 런타임 메모리 소비

7.1.4 테스트 실행 및 자동화

Cargo 테스트 실행

단위 테스트는 표준 Cargo 명령어로 실행됩니다:

```
# 전체 워크스페이스 테스트
```

```
cargo test

# 특정 팰릿 테스트
cargo test -p pallet-pablo

# 벤치마크 실행
cargo test --features runtime-benchmarks
```

Nix 기반 테스트 환경

프로젝트는 Nix를 통해 재현 가능한 테스트 환경을 제공합니다:

```
# flake.nix에서 테스트 관련 설정
{
devShells.default = pkgs.mkShell {
buildInputs = [
rust-toolchain
cargo
# 기타 테스트 도구들
];
};
}
```

테스트 커버리지

테스트 커버리지는 다음과 같은 영역을 포함합니다:

1. **기능 테스트**: 각 extrinsic의 정상 동작 검증
2. **에러 케이스**: 예외 상황 및 에러 처리 검증
3. **경계값 테스트**: 최대/최소값 및 경계 조건 검증
4. **상태 일관성**: 스토리지 상태의 일관성 검증

7.1.5 테스트 품질 보장

테스트 격리

각 테스트는 독립적인 환경에서 실행되어 상호 간섭을 방지합니다:

```
#[test]
fn test_create_pool() {
new_test_ext().execute_with(|| {
// 격리된 테스트 환경에서 실행
assert_ok!(Pablo::create_pool(/* ... */));
});
}
```

어설션 패턴

테스트는 명확한 어설션 패턴을 사용합니다:

```
// 성공 케이스 검증
assert_ok!(Pablo::create_pool(origin, config));
```

```
// 실패 케이스 검증
assert_noop!(
Pablo::create_pool(origin, invalid_config),
Error::<Test>::InvalidConfiguration
);


// 이벤트 검증
assert_has_event!(Event::PoolCreated { pool_id: 1 });
```

테스트 데이터 관리

테스트는 일관된 테스트 데이터를 사용합니다:

```
const ALICE: AccountId = 1;
const BOB: AccountId = 2;
const USDC: AssetId = 100;
const USDT: AssetId = 101;
const INITIAL_BALANCE: Balance = 1_000_000_000_000;
```

이러한 포괄적인 테스트 구조를 통해 Composable Finance 프로젝트는 높은 코드 품질과 시스템 안정성을 유지하며, 복잡한 DeFi 및 크로스체인 기능의 정확성을 보장합니다.